

UNIVERSITY OF SARGODHA
DEPARTMENT OF IT



SUBMITTED BY:

Maheen	[BSIT51F23S015]
Muhammad Zain ul abdin	[BSIT51F23S022]
Taha Bin Tariq	[BSIT51F23S047]
Abdur Rehman	[BSIT51F23S051]

SUBMITTED TO:

Mam Khadija Ilyas

Class :

BSIT (SS1)

LAB

Project :

Online Voting System

ABSTRACT

The **Online Voting System (OVS)** is a secure and efficient software application developed in C++ to modernize the traditional voting process. Voting is a fundamental right and cornerstone of democracy. Historically, elections relied on manual processes involving ballot papers, stamping, and ballot boxes, which were time-consuming, error-prone, and vulnerable to tampering.

The **Online Voting System** eliminates these inefficiencies by digitizing the entire voting process. It replaces manual voting methods with a structured software-based solution that facilitates voter registration, secure login, candidate management, and real-time vote tallying. This system integrates features such as admin and voter authentication, dynamic candidate addition/removal, vote tracking, undo functionality for errors, and leaderboard views.

Leveraging **data structures** such as hash maps, stacks, queues, and sets, the system ensures optimal performance and data integrity. Additionally, the system emphasizes security by requiring voter credentials for access, preventing unauthorized voting, and storing election data persistently for auditability.

This project demonstrates how modern programming techniques can enhance the voting process by offering a scalable, fast, and secure alternative to traditional election systems, thereby ensuring transparency and reliability in democratic practices.

INDEX

CONTENTS	PAGE No.
1. INTRODUCTION	04
2. SOFTWARE REQUIREMENT SPECIFICATION	06
3. SYSTEM DESIGN	08
4. CODING	10
5. SCREENSHOTS	13
6. TIME AND SPACE COMPLEXITY	14
7. CONCLUSION	15

1. Introduction

The **Online Voting System (OVS)** is designed to replicate and enhance the traditional voting process in a user-friendly, efficient, and cost-effective manner using **C++ programming**. This system streamlines the complexities of vote collection, counting, and result analysis by automating the entire process, offering a secure, accurate, and real-time alternative to conventional voting methods. Unlike traditional systems that rely on ballot papers and manual counting—often resulting in delays and errors—this digital solution enables voters to cast their votes electronically, with results calculated instantaneously.

The application incorporates core functionalities such as **voter identification, secure vote collection, vote counting, and dynamic result presentation**. By leveraging C++, the system utilizes **data structures** like hash maps, stacks, and queues to handle voter and candidate data efficiently. This programming language's procedural approach and close interaction with hardware make it ideal for real-time, performance-critical applications like voting systems.

The Need for Computerizing the Voting System

In any organization or democratic setup, managing large volumes of data is critical for effective decision-making, planning, and operational efficiency. Traditional voting methods—while functional—are prone to errors, delays, and inefficiencies due to the reliance on manual processes for voter registration, ballot management, and vote counting. These challenges underline the growing demand for **digital transformation** in voting systems.

Benefits of Computerization:

1. **Speed and Accuracy:** Computers excel in handling repetitive tasks, processing vast amounts of data with minimal errors.
2. **Cost-Effectiveness:** Automated systems reduce the reliance on manual labor, paper ballots, and other physical resources, cutting costs significantly.
3. **Scalability:** As organizations or election setups grow in size and complexity, computerized systems can adapt to handle increasing volumes of data seamlessly.
4. **Transparency and Security:** Computerized voting ensures real-time audit trails, preventing manipulation and enhancing voter confidence in the system.

The Voting System

A **voting system** is a process in which individuals (voters) cast their votes to choose a preferred candidate or decision. The traditional method involves physical ballots, which are counted manually to determine the results. However, this process can be tedious, timeconsuming, and prone to errors.

There are two main types of voting systems:

1. **Open Voting System:** In this system, the identity of the voter is not hidden, and votes are cast openly. This system works well when the number of voters is small, and the stakes are low. However, it lacks privacy and security.
2. **Secret Voting System:** In this system, the voter's identity is concealed, and the vote is kept confidential. This is the more secure and scalable system, allowing for a larger number of voters. The process is typically more system-oriented, and results can be easily counted.

Computer Voting System

The **manual voting system** on a large scale can be expensive and time-consuming, requiring significant resources in terms of manpower, time, and money. By computerizing the voting process, these issues are minimized. The system can handle voting more efficiently, ensuring a faster, more accurate, and cost-effective method of voting.

In the **current manual voting system**, each voter receives a paper ballot, which they use to cast their vote. After voting, the ballots are counted manually, which is a labor-intensive process prone to human error. In contrast, the computerized voting system allows voters to cast votes electronically, which are automatically recorded and counted.

At the end of the voting process, the system can instantly calculate the results, providing accurate data and eliminating the need for physical ballot boxes. This approach not only reduces costs but also ensures the accuracy and transparency of the results.

Problems in Existing System

The **current manual voting system** has several drawbacks:

1. **Expensive:** The traditional system requires significant investment in paper, stationery, and ballot boxes, which increases operational costs.
2. **Time-consuming:** Counting votes manually is a slow process and requires significant time and effort.
3. **Invalid votes:** Many votes become invalid due to mistakes or improper handling of ballots.
4. **Manipulation:** There is a risk of external interference, such as booth capturing or tampering with ballots.
5. **Manpower:** The system requires a large number of personnel to manage and oversee the voting and counting processes.
6. **User Identification Issues:** It is possible for individuals to vote multiple times, leading to incorrect outcomes.
7. **Manipulation in Counting:** The manual counting process can lead to errors or intentional manipulation in vote tallying.
8. **Portability Issues:** After voting, the ballots are transported for counting, posing a risk of theft or mishandling.

Software Program for OVM

Despite the complex design of the election software, which involves interfacing with the control unit processor, there are several security concerns. In this system, we focus on ensuring the integrity and confidentiality of the voting data. The system interface allows voters to vote easily by selecting a candidate on the screen.

The software, developed in C++, is tested in the **Turbo C++ Version 3.0 IDE**. When a user runs the program, it displays a real-time voting machine on the screen with a list of candidates and corresponding buttons. To vote, the user simply clicks on the respective candidate's button. The system also provides an option to view the real-time results by clicking on the "result" button, which displays a graphical bar chart of the votes received by each candidate.

Finally, the user can exit the program by clicking the "exit" button. This intuitive system simplifies the voting process, making it accessible and efficient for users.

2.SYSTEM REQUIREMENT SPECIFICATION (SRS)

The Online Voting System (OVS) is a console-based application designed to digitize the electoral process using the C++ programming language. The system supports voter registration, candidate management, vote casting, and winner declaration. It ensures secure and efficient vote handling while allowing administrators to manage candidates and oversee the voting process.

Functional Requirements

The system supports the following functionalities:

1. Voter Management

- **Sign-Up:** Allows voters to register by providing a unique voter ID and password.
- **Login:** Authenticates voters using their voter ID and password.
- **Vote Casting:** Enables authenticated voters to cast a vote for their preferred candidate.
- **Vote Undo:** Allows voters to undo their last vote if necessary.
- **Prevention of Multiple Voting:** Ensures that voters can only vote once.

2. Candidate Management

- **Add Candidate:** Enables the administrator to add candidates with unique IDs and names.
- **Remove Candidate:** Provides functionality to remove candidates by ID, ensuring only valid IDs are accepted.
- **View Leaderboard:** Displays all candidates with their corresponding vote counts. □
- **Declare Winner:** Identifies and announces the candidate with the highest votes.

3. Admin Panel

- **Admin Login:** Secures access to administrative functions using a predefined username and password.
- **Change Admin Password:** Allows the administrator to update the system's admin password.

4. Data Persistence

- **Save Data:** Automatically stores voter and candidate data into files (`voters.txt` and `candidates.txt`) for future use.
- **Load Data:** Reloads stored voter and candidate data when the system starts.

Non-Functional Requirements

1. Performance

- The system ensures quick responses during file operations and user interactions. □ Designed to handle hundreds of voters and candidates efficiently.

2. Security □ Implements authentication mechanisms for both voters and administrators.

- Restricts voting to authenticated users and prevents multiple votes from the same voter.

3. Reliability

- Ensures consistent system operation with proper handling of invalid inputs.
- Provides clear notifications for errors like duplicate candidates or invalid voter credentials.

4. Usability

- The interface is text-based and simple, ensuring ease of use for all users. □ Step-by-step prompts guide users through various operations.

5. Maintainability □ Modular code structure with functions dedicated to specific tasks allows for easy debugging and future expansion.

- Data files (`voters.txt` and `candidates.txt`) simplify maintenance and data management.

6. Portability

- Operates on any system capable of running a C++ program.
- Relies only on standard C++ libraries, ensuring wide compatibility.

Assumptions and Constraints

1. Assumptions:

- Every voter and candidate has a unique ID.
- The admin password defaults to **admin123** unless changed.

2. Constraints:

- The system requires manual file operations for data transfer between environments.
- Input errors need to be resolved by restarting the operation flow (no autocorrection for invalid inputs).

SYSTEM SPECIFICATIONS

Minimum Hardware Requirements

- **Processor:** Dual-core or above ○ Reason: The application is lightweight but requires a processor that supports standard file operations and multi-threaded tasks.
- **RAM:** 2 GB or above ○ Reason: Sufficient for handling the memory needs of runtime data structures like maps, sets, stacks, and queues.
- **Hard Disk:** 256 GB or above ○ Reason: To store and manage the application, its dependencies, and log files such as **voters.txt** and **candidates.txt**.
- **Keyboard:** Standard or Multimedia Keyboard ○ Reason: Input-heavy application requiring frequent user interactions.

Application Software

- **Programming Language:** C++ ○ Supported IDEs: Turbo C++, Borland C++, Visual Studio, or any modern compiler (GCC/Clang). ○ Reason: The code uses advanced C++ libraries (e.g., STL) for managing complex data structures like **unordered_map**, **set**, and **queue**.
- **Operating System:** Windows 7 or above (Supports backward compatibility to Windows XP/98) ○ Reason: Ensures compatibility with legacy systems and provides a stable runtime for C++ applications.

Additional Specifications

- **Input Devices:** Keyboard (standard or multimedia).
 - **Purpose:** The system is text-driven, requiring manual user input for all operations.
- **Output Devices:** Display screen (monitor) for GUI/CLI-based feedback.
 - **Optional:** Printer for printing voting records or reports.

3. SYSTEM DESIGN

Design Specifications

System Architecture

1. Modules:

- **Admin Panel:** For candidate management, password updates, and vote monitoring.
 - **Voter Panel:** For registration, login, and voting actions.
 - **Data Storage:** File-based system for persisting voters' and candidates' data.
 - **Undo/Redo Mechanism:** Stack-based approach for managing vote reversal.
2. **Flow of Operations:**
- **Admin Workflow:**
 - a. Login with admin credentials.
 - b. Add, remove, or view candidates.
 - c. Declare results or change passwords.
 - d. Save and exit.
 - **Voter Workflow:**
 - a. Register or login using unique credentials.
 - b. Cast a vote or undo a previous vote.
 - c. Logout and return to the main menu.
3. **Data Structures Used:**
- **unordered_map<string, Voter>:** Stores voter credentials and statuses.
 - **unordered_map<string, Candidate>:** Tracks candidates and their vote counts.
 - **set<string>:** Ensures a voter can only vote once.
 - **stack<tuple<string, string>>:** Supports undoing votes.
 - **queue<tuple<string, string>>:** Maintains a sequential log of votes for reporting.
4. **File Management:**
- **voters.txt:** Stores voter credentials and voting statuses (`VoterID`, `Password`, `HasVoted`).
 - **candidates.txt:** Stores candidate details and votes (`CandidateID`, `Name`, `VoteCount`).

System Requirements for Your Code

This specification aligns with the specific code you provided: 1.

Technical Requirements:

- The code uses STL (C++ Standard Template Library), so it must be compiled on a modern C++ compiler.
 - Supports file operations for persisting data, requiring **ifstream** and **ofstream**.
2. **Operational Requirements:**
- Requires manual intervention to register, login, and interact with voting functionalities.
 - Prevents invalid operations through error handling in menu choices.
3. **Security Considerations:**
- Enforces authentication for both voters and admin access.
 - Limits each voter to one vote using `set`.

4. Coding

Data Dictionary

Name	Type	Scope	Description
Voters	unordered_map<string, Voter>	Global	Stores all registered voters, with VoterID as the key and a Voter object containing their details.
Candidates	unordered_map<string, Candidate>	Global	Stores all candidates, with CandidateID as the key and a Candidate object containing their details.
votedVoters	set<string>	Global	Tracks VoterIDs of voters who have already cast their votes to prevent multiple voting.
undoStack	stack<tuple<string, string>>	Global	Stores a tuple of VoterID and CandidateID for undoing the last vote action.
voteQueue	queue<tuple<string, string>>	Global	Maintains a sequential log of votes for reporting or advanced use cases.
adminUsername	string	Global	Stores the username for admin login. Default value is "admin".
adminPassword	string	Global	Stores the password for admin login. Default value is "admin123".
voterID	string	Local	Stores the ID of a voter during login, registration, or voting.
password	string	Local	Stores the password entered by the voter or admin during login or registration.
candidateID	string	Local	Stores the ID of a candidate during operations such as adding, removing, or casting votes.

Name	string	Local	Stores the name of a candidate when registering a new candidate.
newPassword	string	Local	Stores the new password entered by the admin
			when updating their password.
Choice	Int	Local	Stores the user's menu selection in both admin and voter panels.
maxVotes	Int	Local	Tracks the highest vote count during the process of declaring a winner.
winnerID	string	Local	Stores the ID of the candidate with the maximum votes when declaring a winner.

Class Attributes

1.Class: Voter

Attribute Name	Type	Scope	Description
voterID	String	Public	Unique ID of the voter.
Password	String	Public	Password of the voter.
hasVoted	Bool	Public	Indicates whether the voter has cast a vote (true or false).

2.Class: Candidate

Attribute Name	Type	Scope	Description
candidateID	string	Public	Unique ID of the candidate.
Name	string	Public	Name of the candidate.
Votes	Int	Public	Number of votes received by the candidate.

File-Based Data Structures

File Name	Data Stored	Format
voters.txt	Voter details	VoterID Password HasVoted
candidates.txt	Candidate details	CandidateID Name Votes

Global Variables

- **voters:** A map to store all voters, with their voter ID as the key.
- **candidates:** A map to store all candidates, with their candidate ID as the key.
- **votedVoters:** A set that tracks which voters have already voted.
- **undoStack:** A stack that tracks voter actions for undo functionality (stores voter ID and candidate).
- **voteQueue:** A queue that stores votes for processing.
- **adminUsername:** The username for the admin to access the system. □
adminPassword: The password for the admin to access the system.

Function Documentation

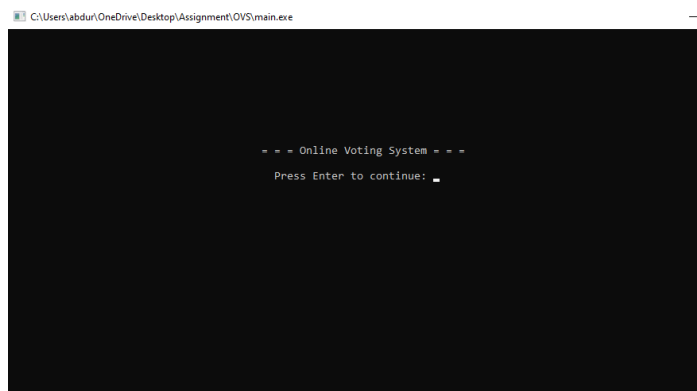
1. **adminMenu()**
Displays the admin menu, providing options for managing the voting system, such as adding/removing candidates and viewing data.
2. **voterMenu()**
Displays the voter menu, allowing voters to cast votes, view the leaderboard, and access other related functionalities.
3. **addCandidate()**
Adds a new candidate to the system with a unique ID and name. Ensures that no duplicate candidates are added.
4. **removeCandidate()**
Removes an existing candidate from the system based on their unique candidate ID.
5. **castVote()**
Allows a voter to cast their vote for a candidate. Updates the candidate's vote count and ensures that a voter can only vote once.
6. **viewLeaderboard()**
Displays the current vote tally for each candidate in a leaderboard format.
7. **undoVote()**
Undoes the last vote cast, allowing the voter to change their vote if needed.
8. **declareWinner()**
Determines the winner based on the highest number of votes. Declares and displays the winner.
9. **saveData()**
Saves the current voting data (voters, candidates, and votes) to a file for persistence between sessions.
10. **loadData()**
Loads voting data from a file, restoring the system's state from the last session.
11. **adminLogin()**
Prompts the admin for their credentials (username and password) and returns true if the login is successful.
12. **voterLogin()**
Prompts the voter for their login credentials and returns true if the login is successful.
13. **voterSignUp()**
Allows a new voter to register by providing a voter ID and password. The voter is then added to the system.

14. **displayVoteHistory()**

Displays the history of all votes cast in the system. If no votes have been cast, a message is shown indicating the same. Iterates through the `voteQueue` to print each voter's ID and their corresponding vote details.

5. Screenshots:

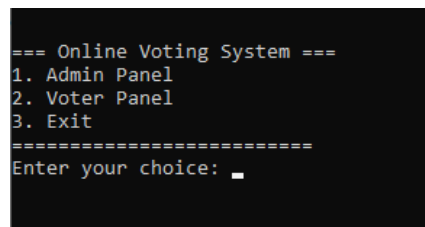
System Introduction



```
C:\Users\abdul\OneDrive\Desktop\Assignment\OVS\main.exe

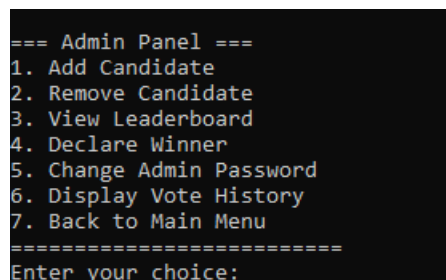
- - - Online Voting System - - -
Press Enter to continue: _
```

Main Menu Option :



```
=== Online Voting System ===
1. Admin Panel
2. Voter Panel
3. Exit
=====
Enter your choice: _
```

Admin Panel :



```
=== Admin Panel ===
1. Add Candidate
2. Remove Candidate
3. View Leaderboard
4. Declare Winner
5. Change Admin Password
6. Display Vote History
7. Back to Main Menu
=====
Enter your choice: _
```

Voter Panel :

```

=== Voter Panel ===
1. Login
2. Sign Up
3. Back to Main Menu
=====
Enter your choice:

```

Voter Actions :

```

=== Voter Actions ===
1. Cast Vote
2. Undo Last Vote
3. Back to Main Menu
=====
Enter your choice:

```

6. Time Complexity:

1. **Admin Login (adminLogin function):**
 - Time Complexity: **O(1)**
The function performs a constant-time check for the admin username and password.
2. **Voter Login (voterLogin function):**
 - Time Complexity: **O(1)**
The function checks if a voter exists in the `voters` map and compares the password, which is done in constant time for each lookup.
3. **Add Candidate (addCandidate function):**
 - Time Complexity: **O(1)**
Inserting a new candidate into the `candidates` map is done in constant time.
4. **Remove Candidate (removeCandidate function):**
 - Time Complexity: **O(1)**
The function uses `erase` on the `candidates` map, which is a constant-time operation on average.
5. **Cast Vote (castVote function):**
 - Time Complexity: **O(1)**
This involves checking if the voter has already voted ($O(1)$ for `votedVoters` set), looking up the candidate in the `candidates` map ($O(1)$ on average), and updating the votes. All of these operations are constant time.
6. **Undo Vote (undoVote function):**
 - Time Complexity: **O(1)**
Undoing a vote involves popping from a stack ($O(1)$ operation) and updating the candidate's votes, which is also an $O(1)$ operation.
7. **View Leaderboard (viewLeaderboard function):**
 - Time Complexity: **O(n)**

This function iterates over all candidates, where n is the number of candidates.

8. **Declare Winner (`declareWinner` function):**

- Time Complexity: **$O(n)$**

Similar to the leaderboard, this function iterates over all candidates to find the one with the maximum votes, where n is the number of candidates.

9. **Save Data (`saveData` function):**

- Time Complexity: **$O(n)$**

Writing data to files involves iterating over the voters, candidates, and voted voters. Therefore, the time complexity is proportional to the total number of voters, candidates, and voted voters.

10. **Load Data (`loadData` function):**

- Time Complexity: **$O(n)$**

Loading data from files involves reading the data for voters, candidates, and voted voters, with each requiring linear time based on the number of entries.

11. **Display Vote History (`displayVoteHistory` function):**

- **Time Complexity:** $O(n)$

Copies the vote queue and iterates through it to print details, where n is the number of votes.

7. Conclusion

The **Online Voting System** exhibits a variety of time complexities depending on the operation. Most of the operations, such as **admin login**, **voter login**, **adding/removing candidates**, **casting votes**, and **undoing votes** have a time complexity of **$O(1)$** , making them highly efficient in execution. However, operations that involve iterating over all candidates, such as **viewing the leaderboard** and **declaring the winner**, have a time complexity of **$O(n)$** , where n is the number of candidates. Additionally, saving and loading data from files involves linear time complexity **$O(n)$** for each data type (voters, candidates, voted voters).

Thus, the overall time complexity is primarily **$O(n)$** in cases where data must be processed or iterated over (e.g., viewing the leaderboard or declaring the winner), and **$O(1)$** for most other actions related to login, vote casting, and management of candidates.

Limitation

1. **Security:** It lacks advanced security features such as encryption and multi-factor authentication.
2. **Data Integrity:** Storing data in text files is prone to corruption and is inefficient for large-scale use.
3. **Scalability:** The system is not optimized for handling large numbers of voters or candidates.
4. **Real-Time Updates:** Results and vote counts are not updated dynamically.

5. **Error Handling:** It lacks comprehensive error-checking and user guidance.
6. **User Interface:** The command-line interface is not user-friendly, especially for nontechnical users.
7. **Audit Trails:** There's no tracking of votes or administrative actions for transparency.
8. **Limited Admin Access:** The system supports only a single admin, limiting management flexibility..

Improvements:

1. **Enhanced Security:** Implement encryption for sensitive data (e.g., voter IDs and passwords), and introduce multi-factor authentication for both admin and voter logins to ensure secure access.
2. **Database Integration:** Use a relational or NoSQL database to replace text files, ensuring better data integrity, scalability, and faster data retrieval.
3. **Real-Time Updates:** Implement real-time vote tallying and leaderboards, possibly through a web-based interface, for up-to-the-minute results.
4. **User Interface:** Develop a graphical user interface (GUI) or web-based application to improve user experience, making it more accessible for non-technical users.
5. **Error Handling and Validation:** Introduce more comprehensive error-handling mechanisms and input validation to prevent invalid inputs and system crashes.
6. **Audit Trails:** Create detailed logs of all actions performed by voters and administrators for transparency, traceability, and accountability.
7. **Scalable Architecture:** Modify the system to support a large number of voters, candidates, and admins, with efficient database queries and server-side processing.
8. **Multiple Admin Support:** Allow multiple administrators with different access levels to manage the system, improving flexibility in administration.

Scope:

1. **Large-Scale Elections:** With improvements, the system could be used for national, regional, or organizational elections, making it scalable for large voter bases.
2. **Online Voting for Organizations:** It could be adapted for businesses, universities, and other organizations to handle internal elections, polls, or decision-making processes.
3. **Blockchain for Transparency:** Blockchain technology could be implemented to ensure transparency and prevent vote tampering, making it suitable for sensitive elections.
4. **Global Reach:** A web-based version of the system could enable online voting globally, providing a secure platform for international elections or events.
5. **Mobile Application:** The system could be extended into a mobile app, allowing voters to participate via smartphones, making it more convenient for widespread use.